

Matplotlib Practice Questions

This notebook covers essential matplotlib plotting techniques including line plots, bar charts, histograms, scatter plots, and multi-line plots.

```
In [42]: # Import required libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

1. Plot $y = x^2$

Create a line plot of the quadratic function $y = x^2$ with proper formatting and annotations.

```
In [43]: # Generate x values
x = np.linspace(-10, 10, 100)
y = x**2

# Create the plot
plt.figure(figsize=(10, 8))
plt.plot(x, y, 'b-', linewidth=2, label='y = x^2')

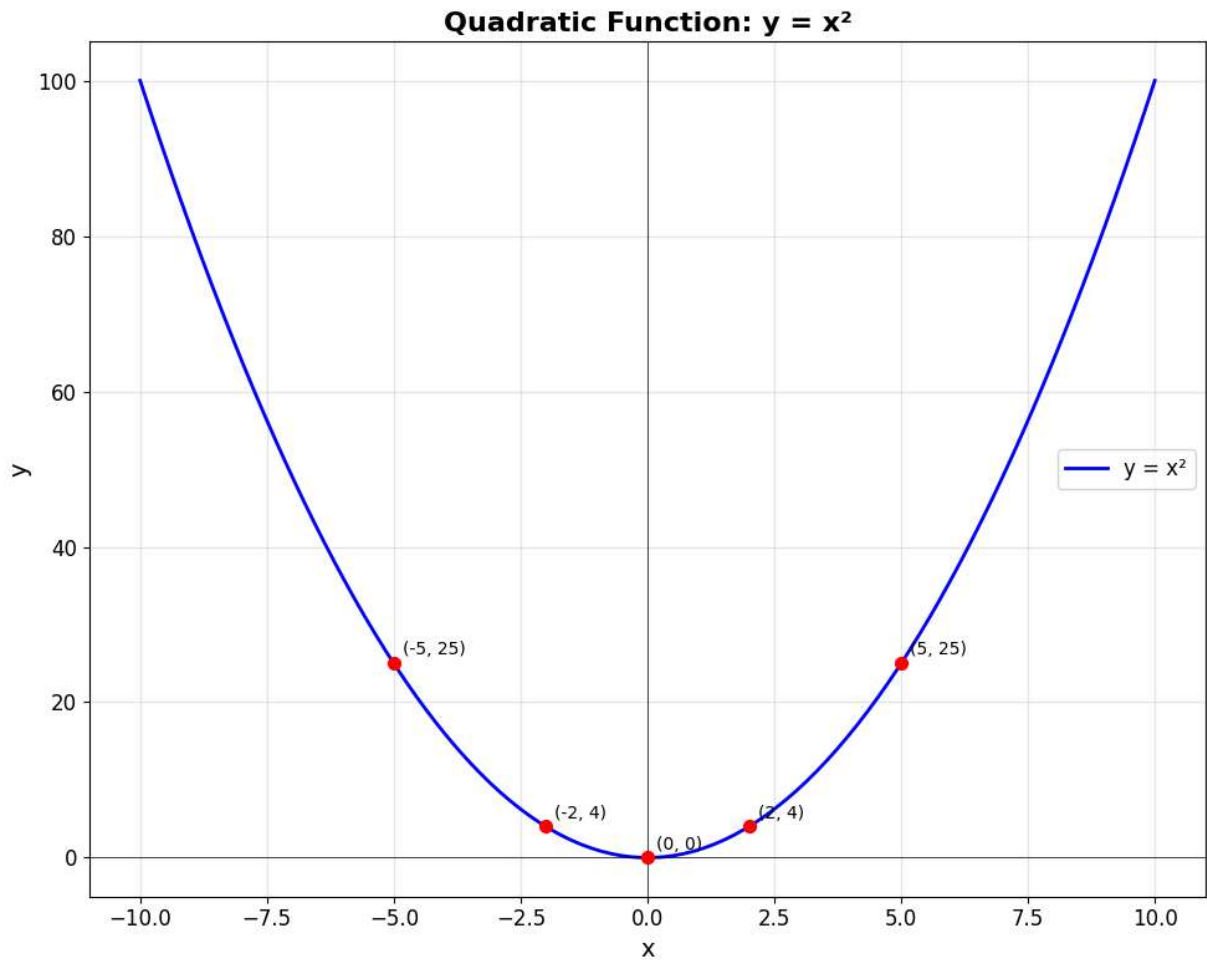
# Add formatting
plt.title('Quadratic Function: y = x^2', fontsize=16, fontweight='bold')
plt.xlabel('x', fontsize=14)
plt.ylabel('y', fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend(fontsize=12)

# Add some special points
special_x = np.array([-5, -2, 0, 2, 5])
special_y = special_x**2
plt.scatter(special_x, special_y, color='red', s=50, zorder=5)

# Annotate special points
for i, (xi, yi) in enumerate(zip(special_x, special_y)):
    plt.annotate(f'({xi}, {yi})', (xi, yi), xytext=(5, 5),
                 textcoords='offset points', fontsize=10)

# Add axis lines through origin
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)

plt.tight_layout()
plt.show()
```



2. Bar Chart of Specific Frequency

Create bar charts showing frequency distribution of different categories with various styling options.

```
In [44]: # Sample data: Programming Languages popularity
languages = ['Python', 'JavaScript', 'Java', 'C++', 'C#', 'PHP', 'Ruby', 'Go']
frequency = [25, 22, 18, 12, 10, 8, 3, 2]

# Create basic bar chart
plt.figure(figsize=(12, 8))
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FFEAA7', '#DDA0DD', '#98D88C']
bars = plt.bar(languages, frequency, color=colors, edgecolor='black', linewidth=1.2)

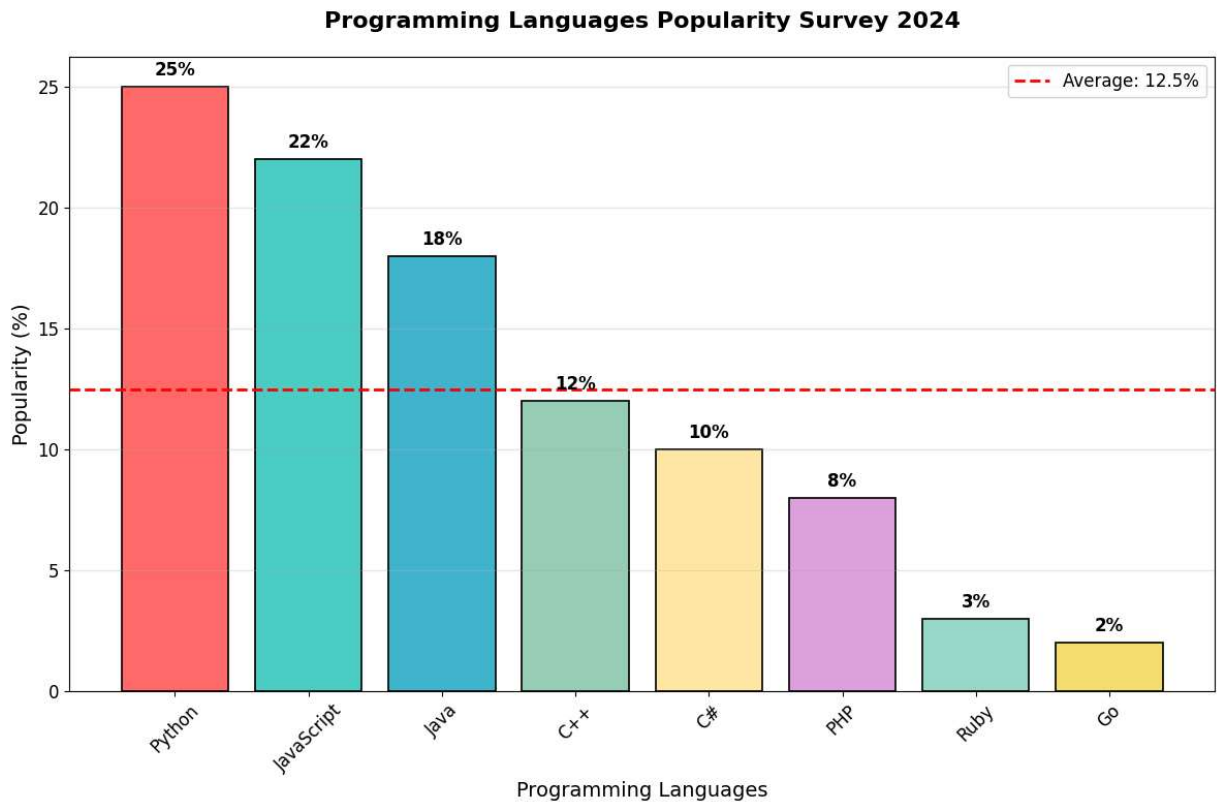
# Add value labels on top of bars
for bar, freq in zip(bars, frequency):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.3,
             f'{freq}%', ha='center', va='bottom', fontweight='bold')

plt.title('Programming Languages Popularity Survey 2024', fontsize=16, fontweight='bold')
plt.xlabel('Programming Languages', fontsize=14)
plt.ylabel('Popularity (%)', fontsize=14)
plt.xticks(rotation=45)
```

```
plt.grid(axis='y', alpha=0.3)

# Add a horizontal line for average
avg_frequency = np.mean(frequency)
plt.axhline(y=avg_frequency, color='red', linestyle='--', linewidth=2,
            label=f'Average: {avg_frequency:.1f}%')
plt.legend()

plt.tight_layout()
plt.show()
```



3. Histogram of Petal Length

Create histograms showing the distribution of petal lengths with different binning strategies and overlays.

```
In [45]: # Generate sample petal length data (simulating iris dataset)
np.random.seed(42)

# Three species with different distributions
setosa_petal = np.random.normal(1.5, 0.2, 50)
versicolor_petal = np.random.normal(4.3, 0.5, 50)
virginica_petal = np.random.normal(5.5, 0.6, 50)

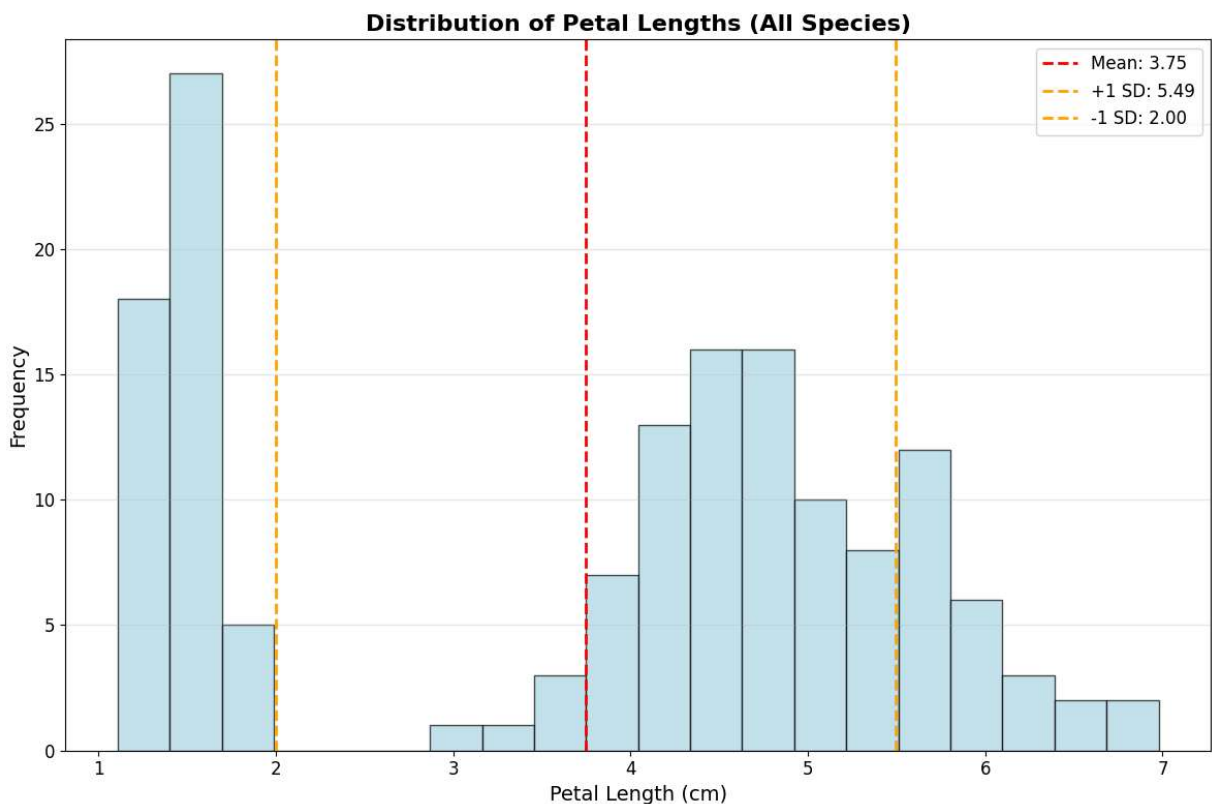
all_petals = np.concatenate([setosa_petal, versicolor_petal, virginica_petal])

# Basic histogram
plt.figure(figsize=(12, 8))
plt.hist(all_petals, bins=20, color='lightblue', edgecolor='black', alpha=0.7)
```

```
plt.title('Distribution of Petal Lengths (All Species)', fontsize=16, fontweight='b')
plt.xlabel('Petal Length (cm)', fontsize=14)
plt.ylabel('Frequency', fontsize=14)
plt.grid(axis='y', alpha=0.3)

# Add statistics
mean_petal = np.mean(all_petals)
std_petal = np.std(all_petals)
plt.axvline(mean_petal, color='red', linestyle='--', linewidth=2, label=f'Mean: {me
plt.axvline(mean_petal + std_petal, color='orange', linestyle='--', linewidth=2, la
plt.axvline(mean_petal - std_petal, color='orange', linestyle='--', linewidth=2, la
plt.legend()

plt.tight_layout()
plt.show()
```



4. Scatter Plot Tree

Create scatter plots showing relationships between tree characteristics with different styling and annotations.

```
In [46]: # Generate sample tree data
np.random.seed(42)
n_trees = 100

# Tree characteristics
tree_height = np.random.normal(15, 5, n_trees) # Height in meters
tree_diameter = 0.3 * tree_height + np.random.normal(0, 1, n_trees) # Diameter in
tree_age = tree_height * 2 + np.random.normal(0, 10, n_trees) # Age in years
```

```

tree_species = np.random.choice(['Oak', 'Pine', 'Maple', 'Birch'], n_trees)

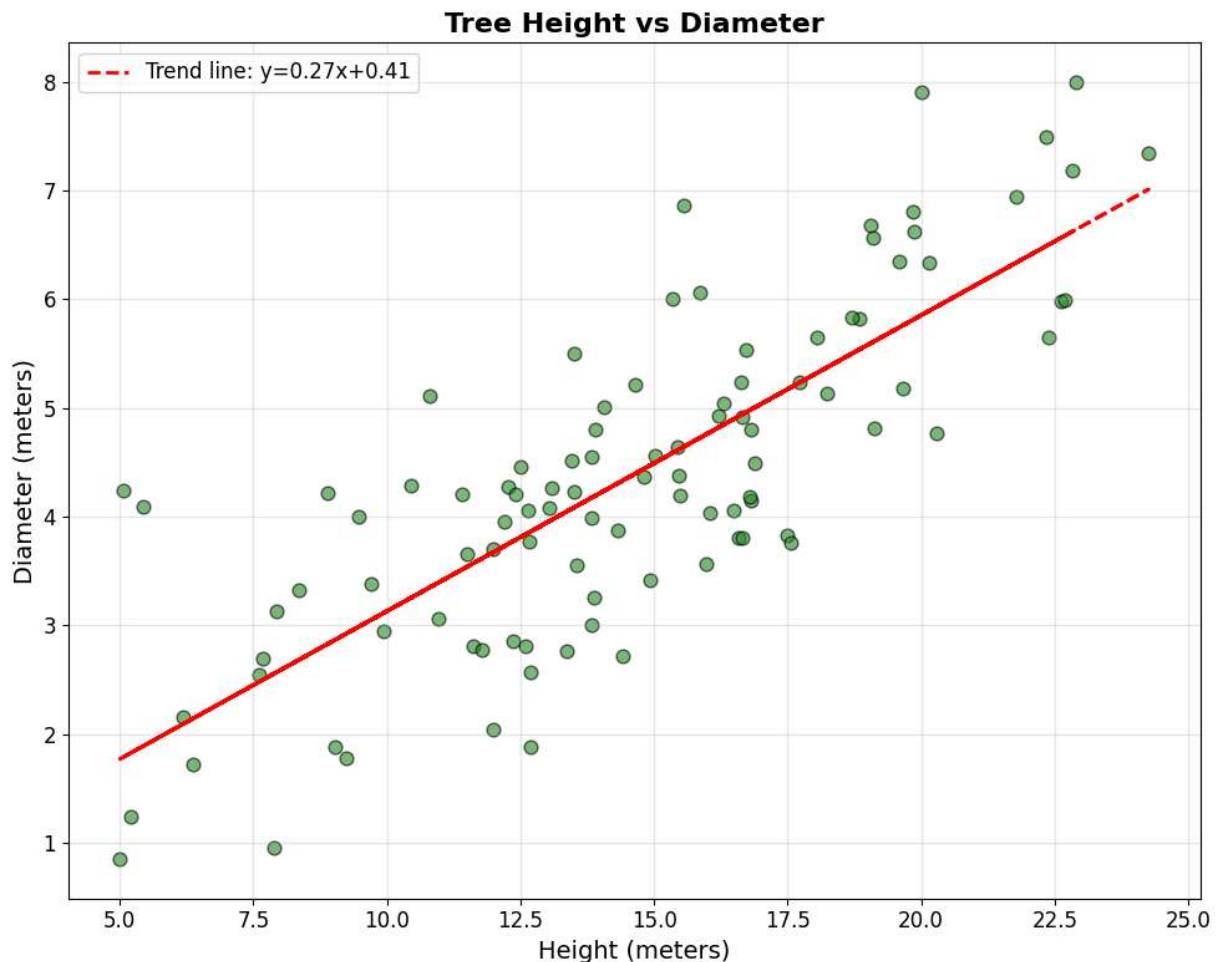
# Ensure positive values
tree_height = np.clip(tree_height, 5, 30)
tree_diameter = np.clip(tree_diameter, 0.5, 8)
tree_age = np.clip(tree_age, 10, 80)

# Basic scatter plot
plt.figure(figsize=(10, 8))
plt.scatter(tree_height, tree_diameter, alpha=0.6, s=60, c='forestgreen', edgecolor='black')
plt.title('Tree Height vs Diameter', fontsize=16, fontweight='bold')
plt.xlabel('Height (meters)', fontsize=14)
plt.ylabel('Diameter (meters)', fontsize=14)
plt.grid(True, alpha=0.3)

# Add trend line
z = np.polyfit(tree_height, tree_diameter, 1)
p = np.poly1d(z)
plt.plot(tree_height, p(tree_height), "r--", linewidth=2, label=f'Trend line: y={z[0]x+z[1]}')
plt.legend()

plt.tight_layout()
plt.show()

```



5. Plot Multiple Lines

Create line plots with multiple series, different styles, and comprehensive formatting.

```
In [47]: # Generate sample data for multiple time series
x = np.linspace(0, 10, 100)

# Mathematical functions
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.sin(2*x)
y4 = np.exp(-x/3) * np.sin(x)

# Basic multiple lines plot
plt.figure(figsize=(12, 8))
plt.plot(x, y1, 'b-', linewidth=2, label='sin(x)')
plt.plot(x, y2, 'r--', linewidth=2, label='cos(x)')
plt.plot(x, y3, 'g:', linewidth=3, label='sin(2x)')
plt.plot(x, y4, 'm-.', linewidth=2, label='e^(-x/3) * sin(x)')

plt.title('Multiple Mathematical Functions', fontsize=16, fontweight='bold')
plt.xlabel('x', fontsize=14)
plt.ylabel('y', fontsize=14)
plt.legend(fontsize=12, loc='upper right')
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)

plt.tight_layout()
plt.show()

# Time series data (simulated stock prices)
np.random.seed(42)
days = np.arange(0, 365)
base_price = 100

# Simulate different stock trends
stock_a = base_price + np.cumsum(np.random.normal(0.1, 1, 365)) # Upward trend
stock_b = base_price + np.cumsum(np.random.normal(-0.05, 1.2, 365)) # Downward trend
stock_c = base_price + np.cumsum(np.random.normal(0.05, 0.8, 365)) # Stable
stock_d = base_price + 10*np.sin(days/30) + np.cumsum(np.random.normal(0, 0.5, 365))

plt.figure(figsize=(14, 8))
plt.plot(days, stock_a, linewidth=2, label='Tech Stock A', color='blue')
plt.plot(days, stock_b, linewidth=2, label='Energy Stock B', color='red')
plt.plot(days, stock_c, linewidth=2, label='Utility Stock C', color='green')
plt.plot(days, stock_d, linewidth=2, label='Retail Stock D', color='orange')

plt.title('Stock Price Movement Over One Year', fontsize=16, fontweight='bold')
plt.xlabel('Days', fontsize=14)
plt.ylabel('Stock Price ($)', fontsize=14)
plt.legend(fontsize=12, loc='upper left')
plt.grid(True, alpha=0.3)

# Add some annotations
max_a_idx = np.argmax(stock_a)
plt.annotate(f'Peak A: ${stock_a[max_a_idx]:.2f}',
```

```

        xy=(max_a_idx, stock_a[max_a_idx]),
        xytext=(max_a_idx+30, stock_a[max_a_idx]+10),
        arrowprops=dict(arrowstyle='->', color='blue'),
        fontsize=10, color='blue')

plt.tight_layout()
plt.show()

# Temperature data throughout the day
hours = np.arange(0, 24)
temp_summer = 25 + 8*np.sin(np.pi*(hours-6)/12) + np.random.normal(0, 1, 24)
temp_winter = 5 + 6*np.sin(np.pi*(hours-6)/12) + np.random.normal(0, 1.5, 24)
temp_spring = 15 + 7*np.sin(np.pi*(hours-6)/12) + np.random.normal(0, 1.2, 24)
temp_autumn = 12 + 5*np.sin(np.pi*(hours-6)/12) + np.random.normal(0, 1, 24)

plt.figure(figsize=(12, 8))
plt.plot(hours, temp_summer, 'o-', linewidth=2, markersize=6, label='Summer', color='red')
plt.plot(hours, temp_winter, 's-', linewidth=2, markersize=6, label='Winter', color='blue')
plt.plot(hours, temp_spring, '^-', linewidth=2, markersize=6, label='Spring', color='green')
plt.plot(hours, temp_autumn, 'd-', linewidth=2, markersize=6, label='Autumn', color='purple')

plt.title('Temperature Variation Throughout the Day by Season', fontsize=16, fontweight='bold')
plt.xlabel('Hour of Day', fontsize=14)
plt.ylabel('Temperature (°C)', fontsize=14)
plt.legend(fontsize=12)
plt.grid(True, alpha=0.3)
plt.xticks(range(0, 25, 3))

# Add shaded regions for night time
plt.axvspan(0, 6, alpha=0.2, color='navy', label='Night')
plt.axvspan(20, 24, alpha=0.2, color='navy')

plt.tight_layout()
plt.show()

# Subplots with multiple lines
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Trigonometric functions
x = np.linspace(0, 4*np.pi, 100)
axes[0, 0].plot(x, np.sin(x), 'b-', label='sin(x)')
axes[0, 0].plot(x, np.cos(x), 'r-', label='cos(x)')
axes[0, 0].plot(x, np.tan(x), 'g-', label='tan(x)')
axes[0, 0].set_title('Trigonometric Functions')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].set_ylim(-3, 3)

# Exponential functions
x = np.linspace(0, 3, 100)
axes[0, 1].plot(x, np.exp(x), 'b-', label='e^x')
axes[0, 1].plot(x, np.exp(-x), 'r-', label='e^(-x)')
axes[0, 1].plot(x, np.exp(2*x), 'g-', label='e^(2x)')
axes[0, 1].set_title('Exponential Functions')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

```

```

# Logarithmic functions
x = np.linspace(0.1, 10, 100)
axes[1, 0].plot(x, np.log(x), 'b-', label='ln(x)')
axes[1, 0].plot(x, np.log10(x), 'r-', label='log10(x)')
axes[1, 0].plot(x, np.log2(x), 'g-', label='log2(x)')
axes[1, 0].set_title('Logarithmic Functions')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Polynomial functions
x = np.linspace(-2, 2, 100)
axes[1, 1].plot(x, x, 'b-', label='x')
axes[1, 1].plot(x, x**2, 'r-', label='x^2')
axes[1, 1].plot(x, x**3, 'g-', label='x^3')
axes[1, 1].plot(x, x**4, 'm-', label='x^4')
axes[1, 1].set_title('Polynomial Functions')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].set_ylim(-5, 5)

plt.tight_layout()
plt.show()

```

